САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа № 4
«Развёртывание приложения на удалённом сервере»

Выполнил:
Машковцева Марина
БР1.2

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Провести предварительную настройку удалённого сервера, развернуть приложение и настроить Nginx как обратный прокси для обеспечения внешнего доступа.

Ход работы

1. Подготовка виртуальной машины

Для размещения бэкенд-приложения создана виртуальная машина recipe-api-server в Yandex Cloud (Compute Cloud). Установлена Ubuntu 24.04 LTS. Конфигурация: 2 vCPU, 4 ГБ оперативной памяти и диск 20 ГБ. Серверу назначен публичный IPv4-адрес 158.160.183.219.

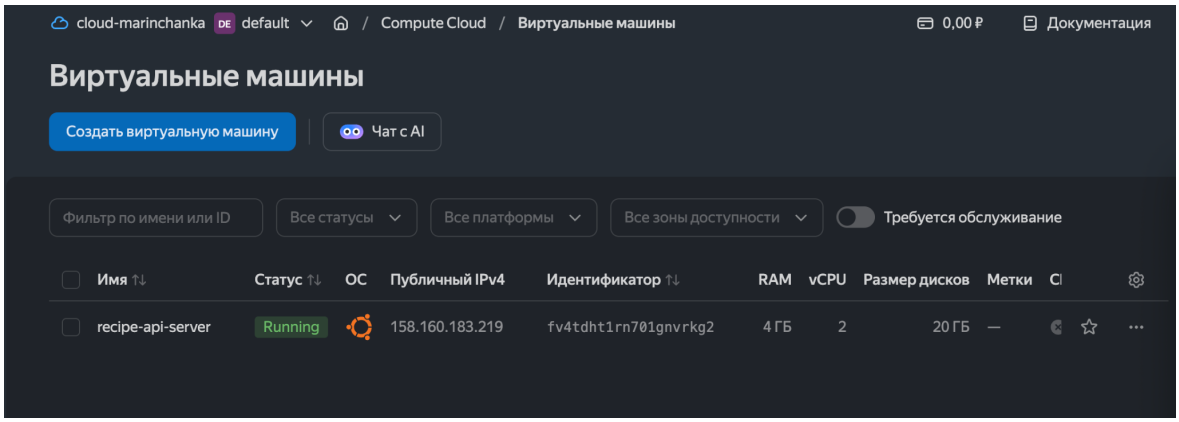


Рисунок 1 – Виртуальная машина в панели Yandex Cloud

Подключение выполнено по SSH с использованием ключа ED25519. Перед установкой программного обеспечения обновлён список доступных пакетов.

```
ssh -i ~/.ssh/id_ed25519 ubuntu@158.160.183.219
sudo apt update
```



Рисунок 2 – Подключение к Ubuntu по SSH

2. Установка и проверка зависимостей

На сервере установлены Git, Docker, Docker Compose и Nginx. Docker используется для контейнеризации сервисов, Compose — для совместного запуска компонентов, Git — для получения исходного кода, Nginx — для обработки внешних HTTP-запросов.

```
docker --version
docker compose version
```

Проверены версии Docker 29.1.3 и Docker Compose 2.40.3. На момент проверки до запуска приложения список контейнеров был пуст.

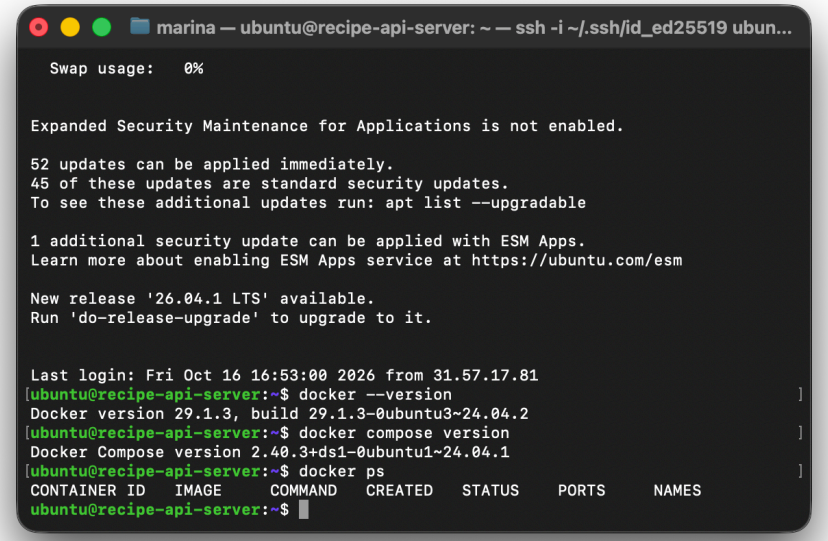


Рисунок 3 – Проверка Docker и Docker Compose на сервере

3. Подготовка файлов приложения

Из ветки feature/lab4 репозитория ITMO-ACS-Backend-2026-A на сервер получен каталог labs/lab4/microservices-recipe. В нём находятся исходный код четырёх Node.js-сервисов, файлы Dockerfile, docker-compose.yml, конфигурация nginx/recipe-api.conf, .env.example и README.md.

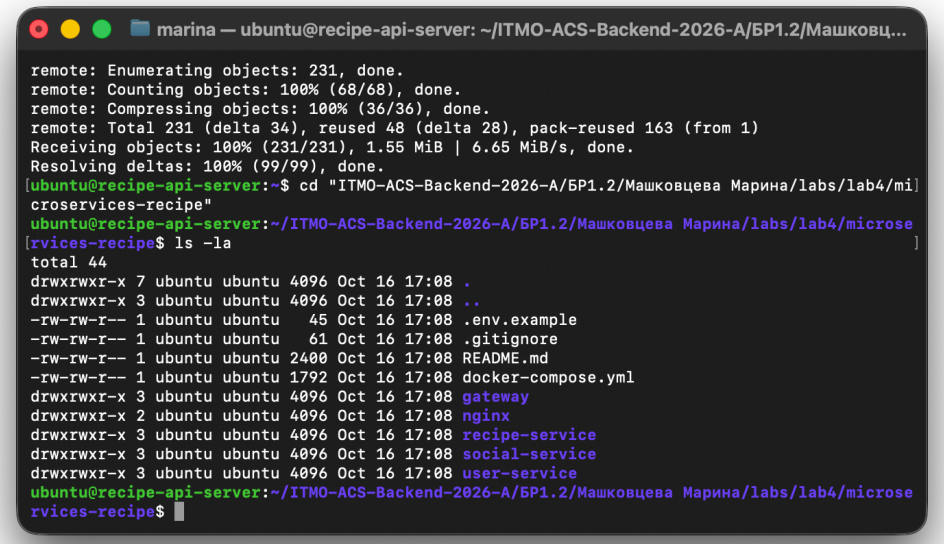


Рисунок 4 – Файлы проекта после клонирования

В docker-compose.yml определены контейнеры gateway, user-service, recipe-service, social-service и RabbitMQ. Dockerfile каждого сервиса описывает сборку соответствующего образа. Для приложения создан локальный .env с JWT_SECRET; этот файл не публикуется в Git и не включается в отчёт.

```
docker compose up -d --build
docker compose ps
```

Команда собирает образы и запускает все компоненты в фоновом режиме. API Gateway доступен на сервере через 127.0.0.1:3000, а RabbitMQ обеспечивает взаимодействие микросервисов.

4. Настройка Nginx

Конфигурационный файл nginx/recipe-api.conf задаёт прослушивание порта 80 и перенаправление входящих запросов к API Gateway. Передаются исходный Host, IP клиента и протокол обращения.

```
server {
    listen 80;
    server_name _;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Конфигурация скопирована в sites-available, активирована в sites-enabled, а стандартный сайт отключён. После проверки синтаксиса конфигурация Nginx была перезагружена.

```
sudo nginx -t
sudo systemctl reload nginx
```

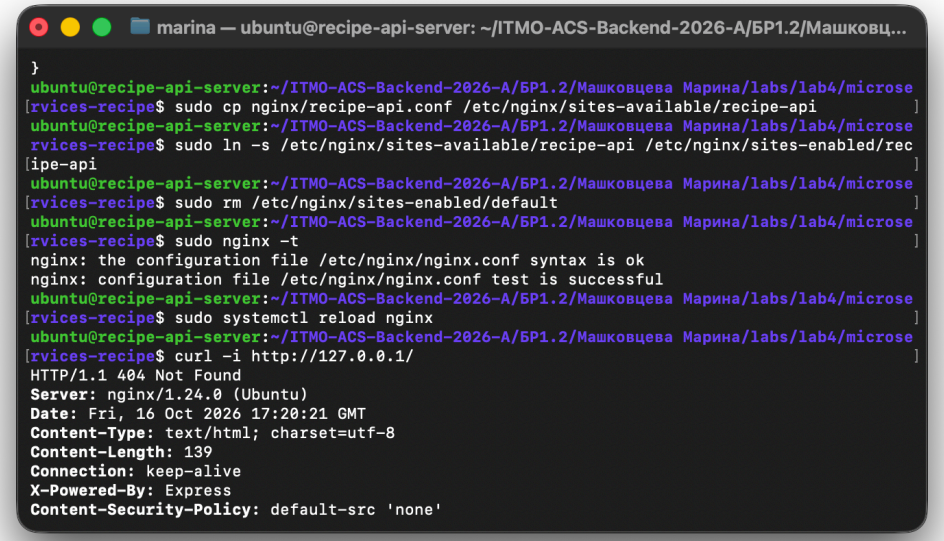


Рисунок 5 – Проверка конфигурации и перезагрузка Nginx

5. Проверка доступности приложения

С локального компьютера выполнены HTTP-запросы на публичный IP сервера. Эндпоинт /health проверяет работу API Gateway, а /api/recipes возвращает список рецептов.

```
curl -i http://158.160.183.219/health
curl -i http://158.160.183.219/api/recipes
```

Оба запроса вернули HTTP/1.1 200 OK. Для /health получен JSON {"status":"ok","service":"gateway"}, для /api/recipes — пустой массив []. Отсутствие записей в списке рецептов является ожидаемым результатом.

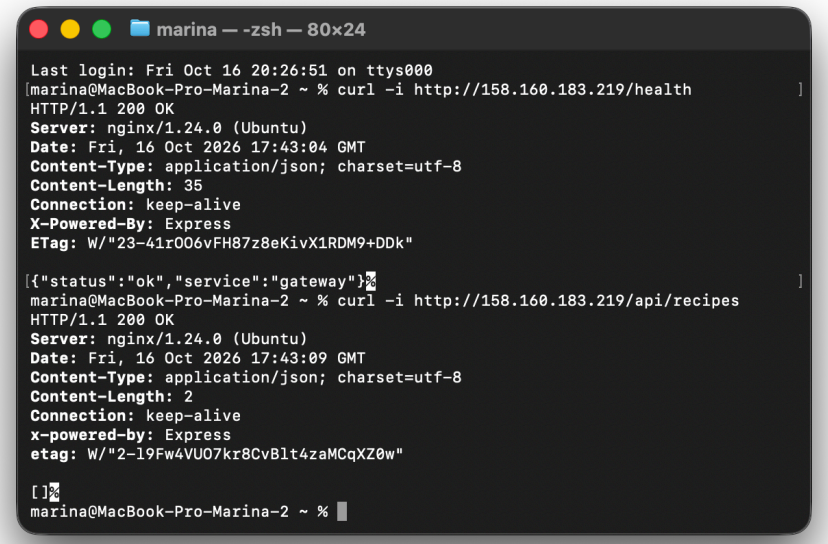


Рисунок 6 – Успешные запросы к API через публичный адрес

Вывод

В ходе работы настроен сервер Ubuntu, развёрнуто микросервисное приложение в Docker и обеспечен внешний доступ к API через Nginx. Запросы к двум конечным точкам завершились успешно со статусом HTTP 200 OK.